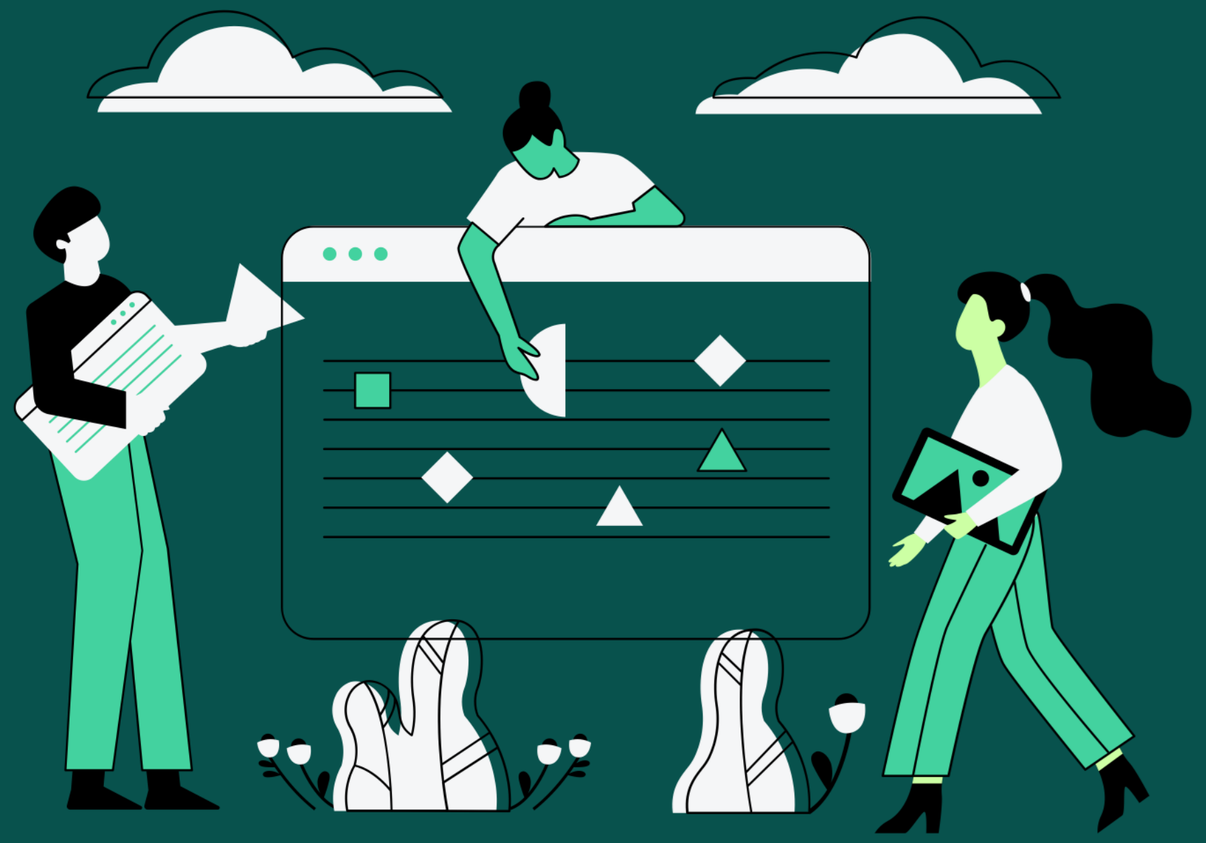
SmartQueryBI

Mehrschichtige Caching-Architekturen in modernen BI-Systemen

Torsten Steinmüller

Kamal Badawi

TABLE OF CONTENTS



01 Idee

02 Caching-Strategien

03 Fazit

01

IDEE

Von Text zu Diagramm

Nutzerinput

Datenwunsch in Textform eingeben

Analyse

Gemini interpretiert Text → SQL-Abfrage & passendes Nivo.js-Diagramm

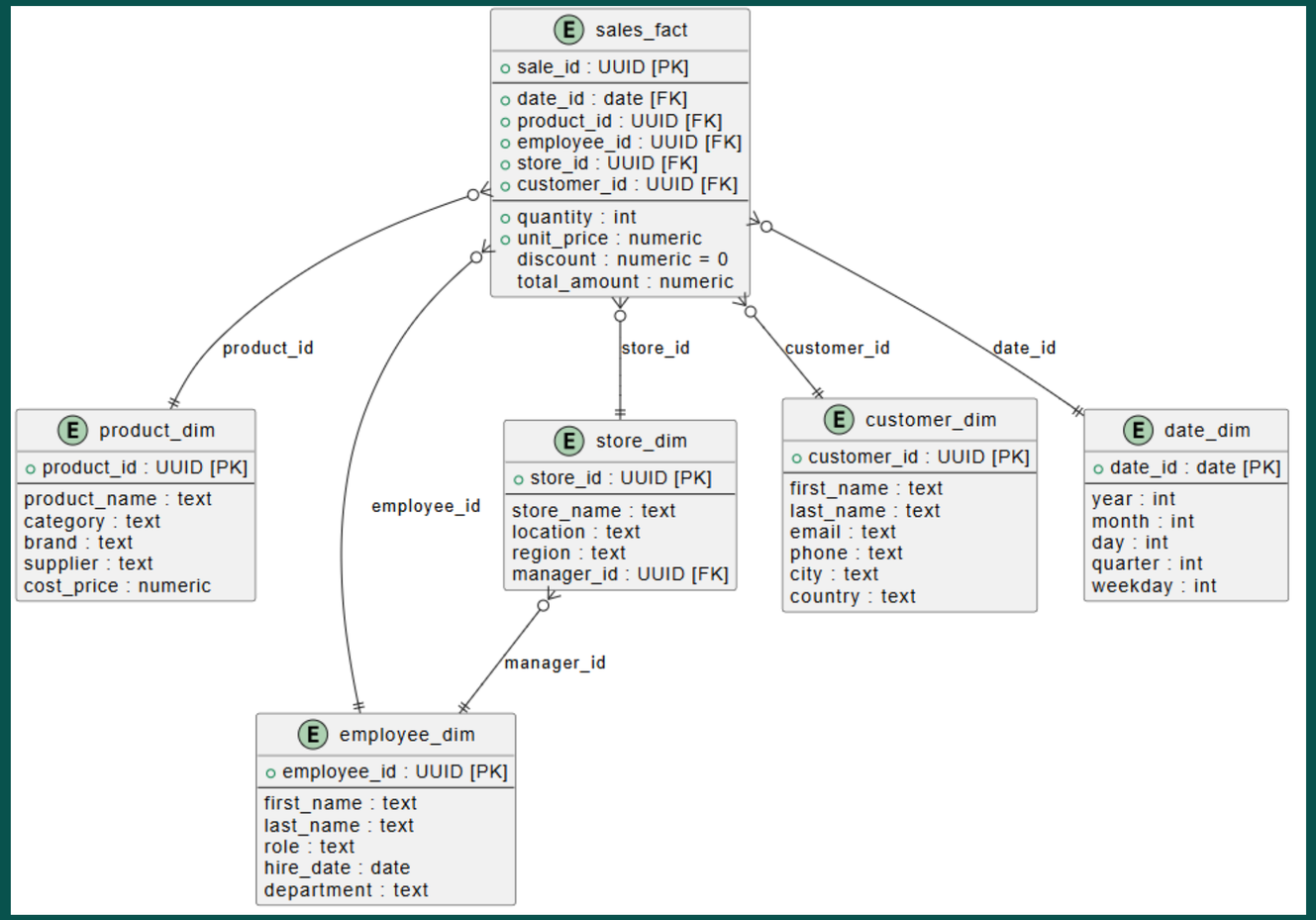
Datenabruf

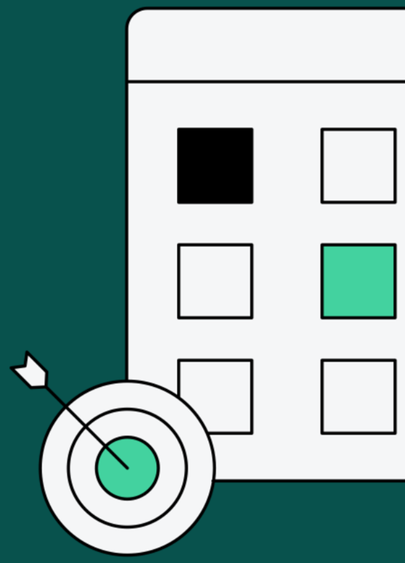
SQL-Abfrage zieht Daten aus Supabase

Aufbereitung

Daten werden für das gewählte Diagramm formatiert

ER-Modell


sales_fact
sale_id : UUID [PK]
date_id : date [FK]
product_id : UUID [FK]
employee_id : UUID [FK]
store_id : UUID [FK]
customer_id : UUID [FK]
quantity : int
unit_price : numeric
discount : numeric = 0
total_amount : numeric
product_id
store_id
customer_id
date_id
employee_id
manager_id
product_dim
product_id : UUID [PK]
product_name : text
category : text
brand : text
supplier : text
cost_price : numeric
store_dim
store_id : UUID [PK]
store_name : text
location : text
region : text
manager_id : UUID [FK]
customer_dim
customer_id : UUID [PK]
first_name : text
last_name : text
email : text
phone : text
city : text
country : text
date_dim
date_id : date [PK]
year : int
month : int
day : int
quarter : int
weekday : int
employee_dim
employee_id : UUID [PK]
first_name : text
last_name : text
role : text
hire_date : date
department : text



Herausforderungen

01

Hohe Wartezeiten

Abfragen dauern lange

02

Hohe Betriebskosten

Viele Datenabfragen
belasten Ressourcen

03

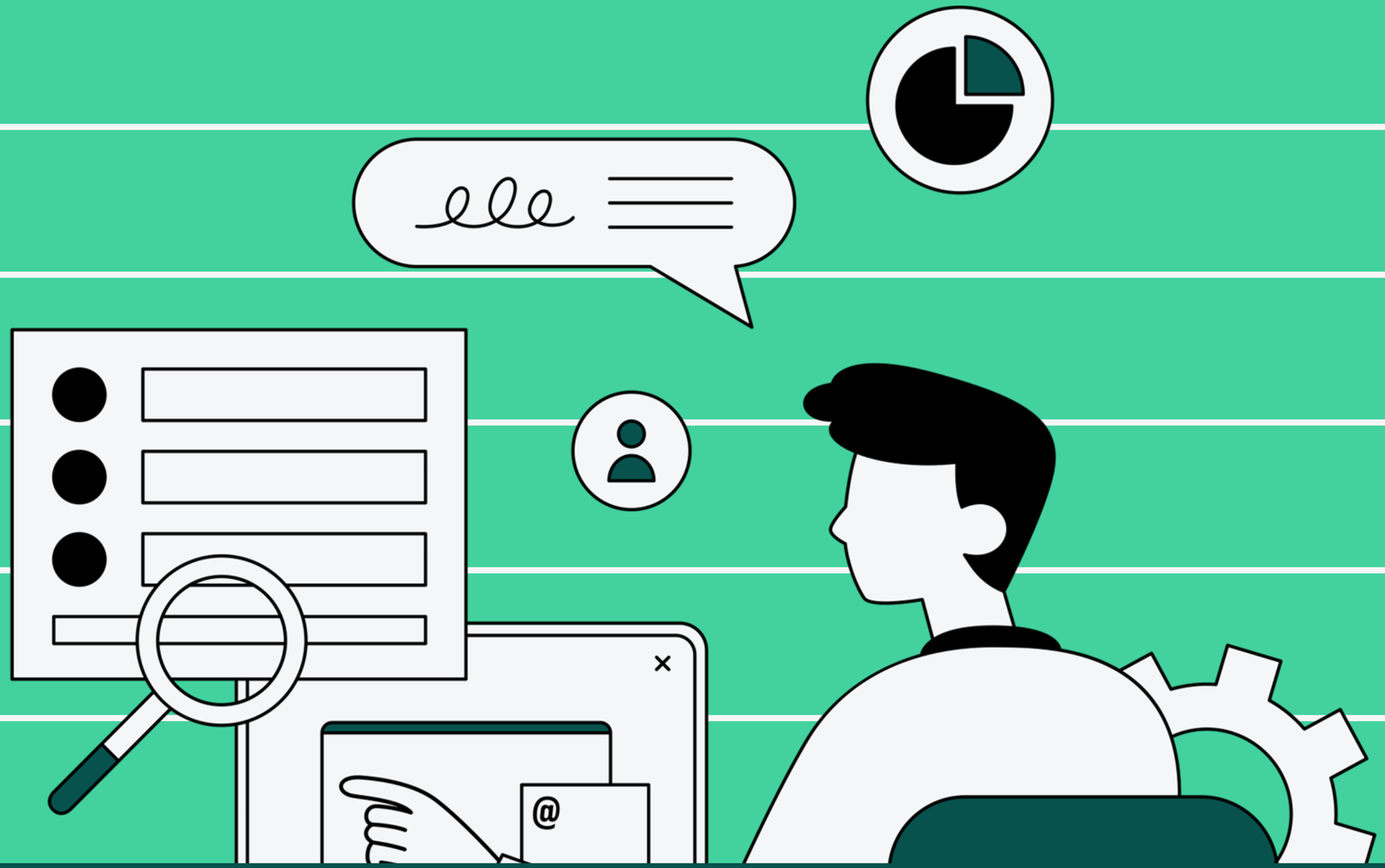
Skalierungsprobleme

Systemleistung sinkt
bei vielen Nutzern

Lösung

Caching

Zwischenspeicherung von Daten für schnellere Abfragen, geringere Kosten und stabile Performance auch bei vielen Nutzern



02 Caching-Strategien

Client-Side Caching

Funktionsweise

- Antworten auf identische Anfragen werden lokal gespeichert
- Nutzung von HTTP-Caching (z. B. Cache-Control)
- Entlastet Server & Datenbank

Einsatzszenarien

- Standardberichte & KPIs
- Interaktive Web-Dashboards
- Wiederholte Visualisierungen

Vorteile

- Extrem niedrige Latenz (sub-ms)
- Keine Serverressourcen nötig
- Ideal für wiederholte Dashboard-Abfragen

Nachteile

- Nur für identische Anfragen wirksam
- Personalisierte Daten eingeschränkt cache-bar
- Kaum Kontrolle über Browser-Cache

Server-Side LRU Cache

Funktionsweise

- Speichert häufig genutzte Ergebnisse im RAM
- LRU-Prinzip: alte Einträge werden automatisch entfernt
- Ideal für wiederholte serverseitige SQL-/LLM-Pipelines

Einsatzszenarien

- Ad-hoc-Analysen
- Wiederkehrende Abfragen auf einem Server
- Vorhalten populärer Reports im RAM

Vorteile

- Millisekunden-Antwortzeiten
- Kein externer Speicher notwendig
- Sehr geringer Implementierungsaufwand

Nachteile

- Keine Persistenz
- Nur für eine einzelne Serverinstanz
- Speicher begrenzt (maxsize)

Redis Distributed Cache

Funktionsweise

- Externer, persistenter Key-Value Store
- Multi-Server- und Multi-User-fähig
- Beinhaltet vollständige Pipelines (LLM → SQL → Daten → Nivo-Dataset)

Einsatzszenarien

- Enterprise-Dashboards
- Multi-Instance Deployments
- Häufig genutzte, persistente BI-Queries

Vorteile

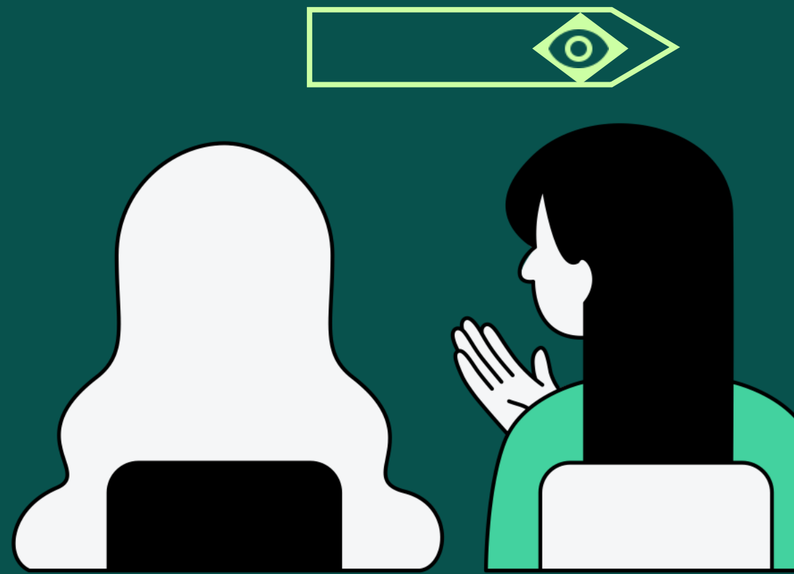
- Hohe Skalierbarkeit
- Persistente Zwischenspeicherung
- Deutliche Reduktion von LLM- und Datenbank-Aufrufen

Nachteile

- Höhere Komplexität (Deployment & Überwachung)
- Netzwerkabhängige Latenz
- Aktive Cache-Invalidation notwendig

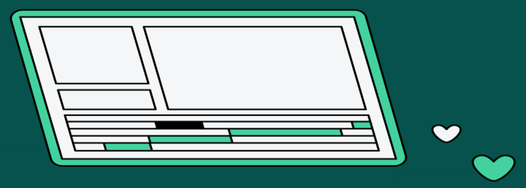
Vergleich

Eigenschaft	Client	Server LRU	Redis
Ort	Browser / CDN	Server RAM	Externer Store
Persistenz	Kurzlebig	Nein	Ja
Latenz	sub-ms	ms	1–10 ms
Skalierbarkeit	pro Client	Single Server	Multi-Server
Vorteile	extrem schnell	schnell, gering	persistent, verteilt
Nachteile	Datenlecks	kein Multi-Server	Netzwerk, Setup



03 Fazit

Effiziente BI mit mehrschichtigem Caching



Balance zwischen Aktualität & Performance

Schutz personenbezogener Daten

Datenänderungen berücksichtigen

Performanceproblem e früh erkennen

Performance steigern

Millisekunden- bis Sub-Millisekunden-Antwortzeiten durch Caching

Skalierbarkeit erhöhen

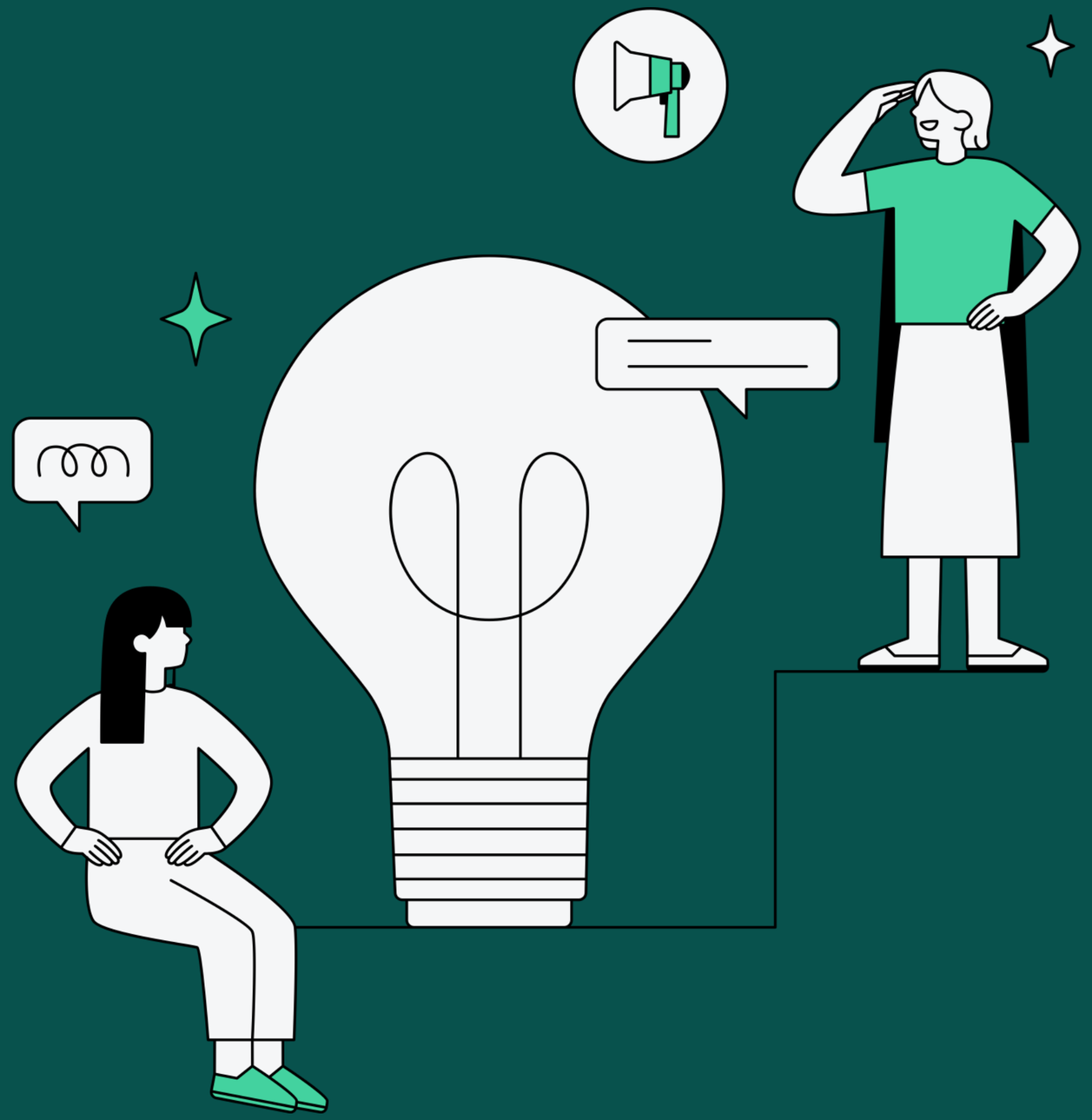
Horizontale Erweiterung & Reduzierung der Datenbanklast

Kosten senken

Weniger LLM-API-Aufrufe, optimierte Serverressourcen

Benutzerfreundlichkeit

Interaktive Dashboards ohne Wartezeiten



THANK YOU